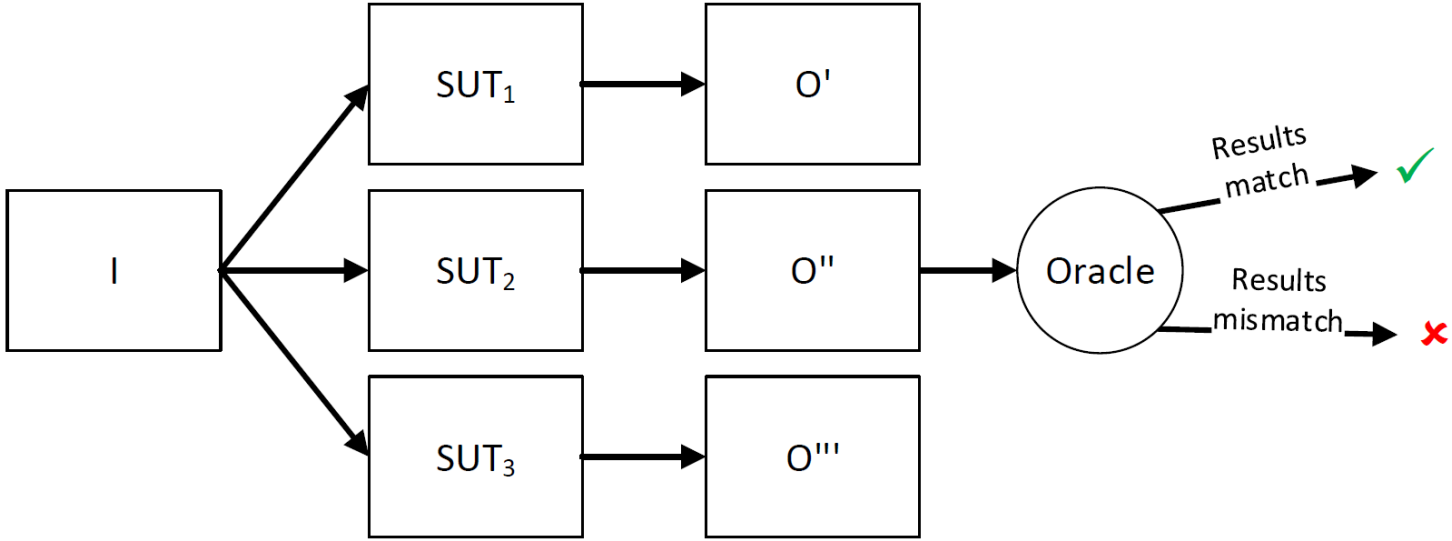
CS3213: Foundations of Software Engineering

Metamorphic Testing

© Copyright National University of Singapore. All Rights Reserved.

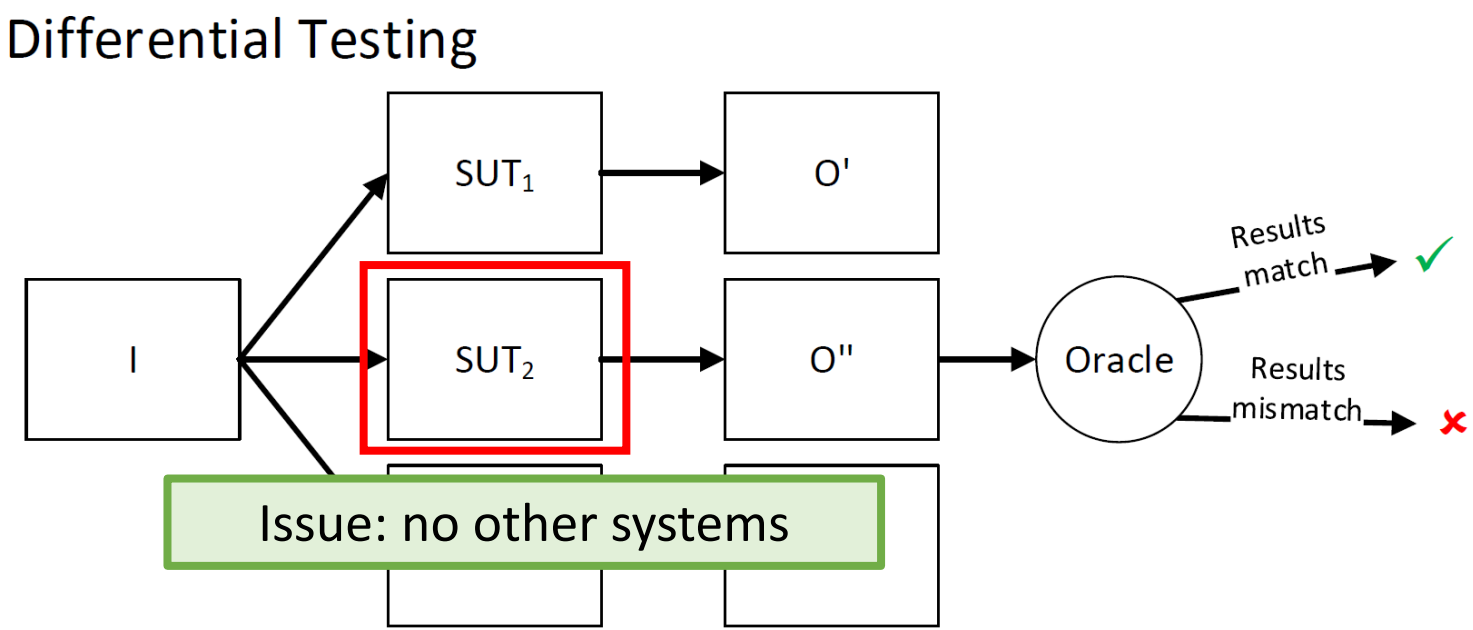
Differential Testing

I: Input
O: Output
SUT: Software Under Test



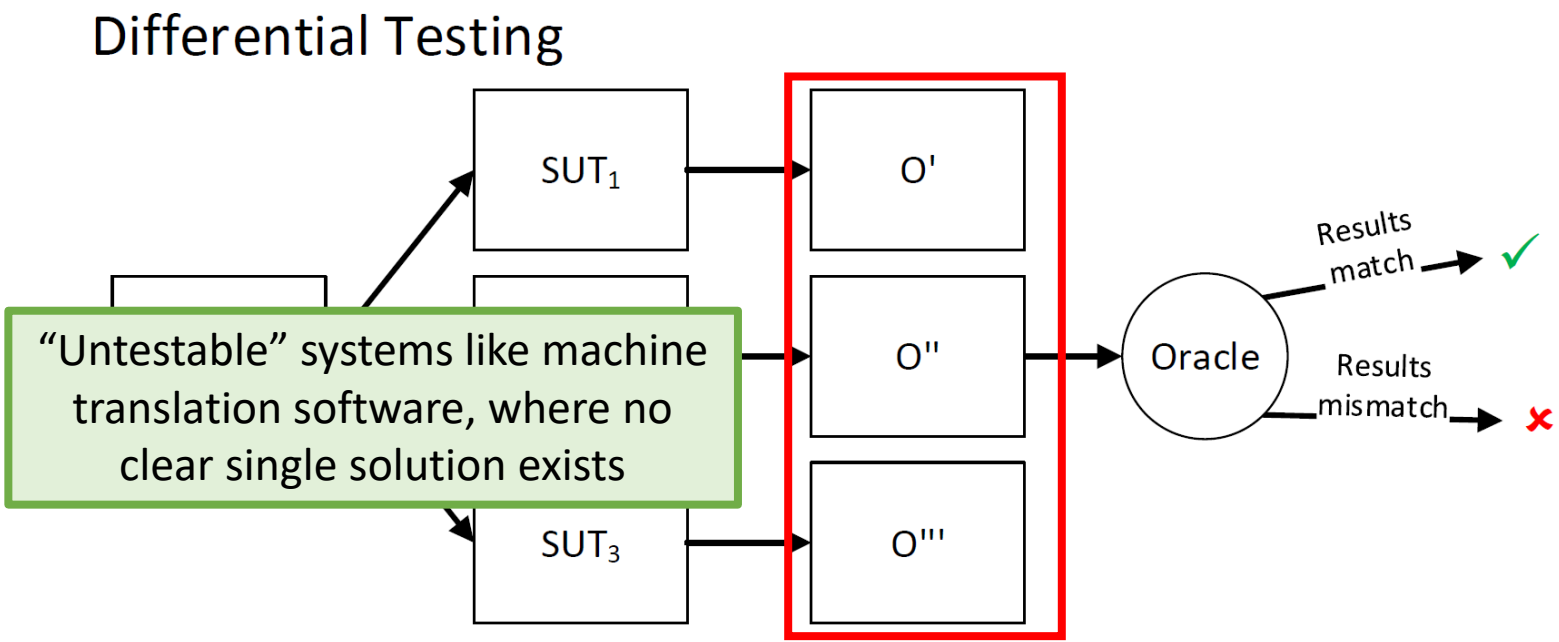
Differential Testing

I: Input
O: Output
SUT: Software Under Test



Differential Testing

I: Input
O: Output
SUT: Software Under Test

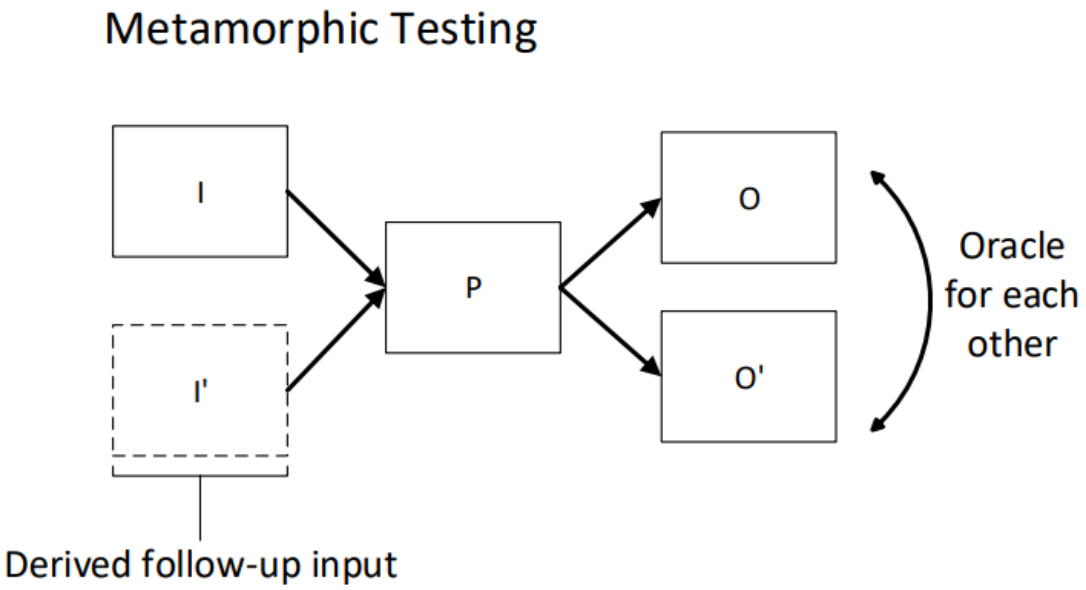


Metamorphic Testing

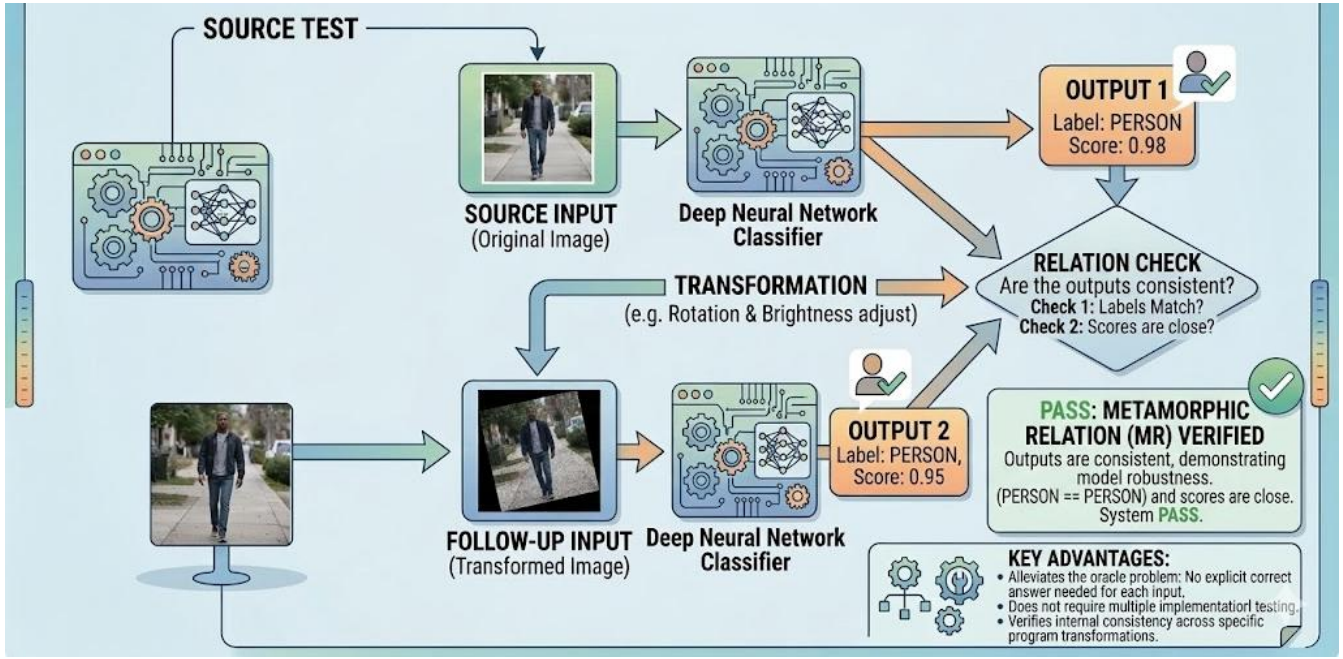
- ***Metamorphic testing***: use a source test input (and its result) to generate a *follow-up test input* for which the result is known
  - Involves a *metamorphic relation*
- Black-box approach

Metamorphic Testing

I: Input
O: Output
P: Software Under Test

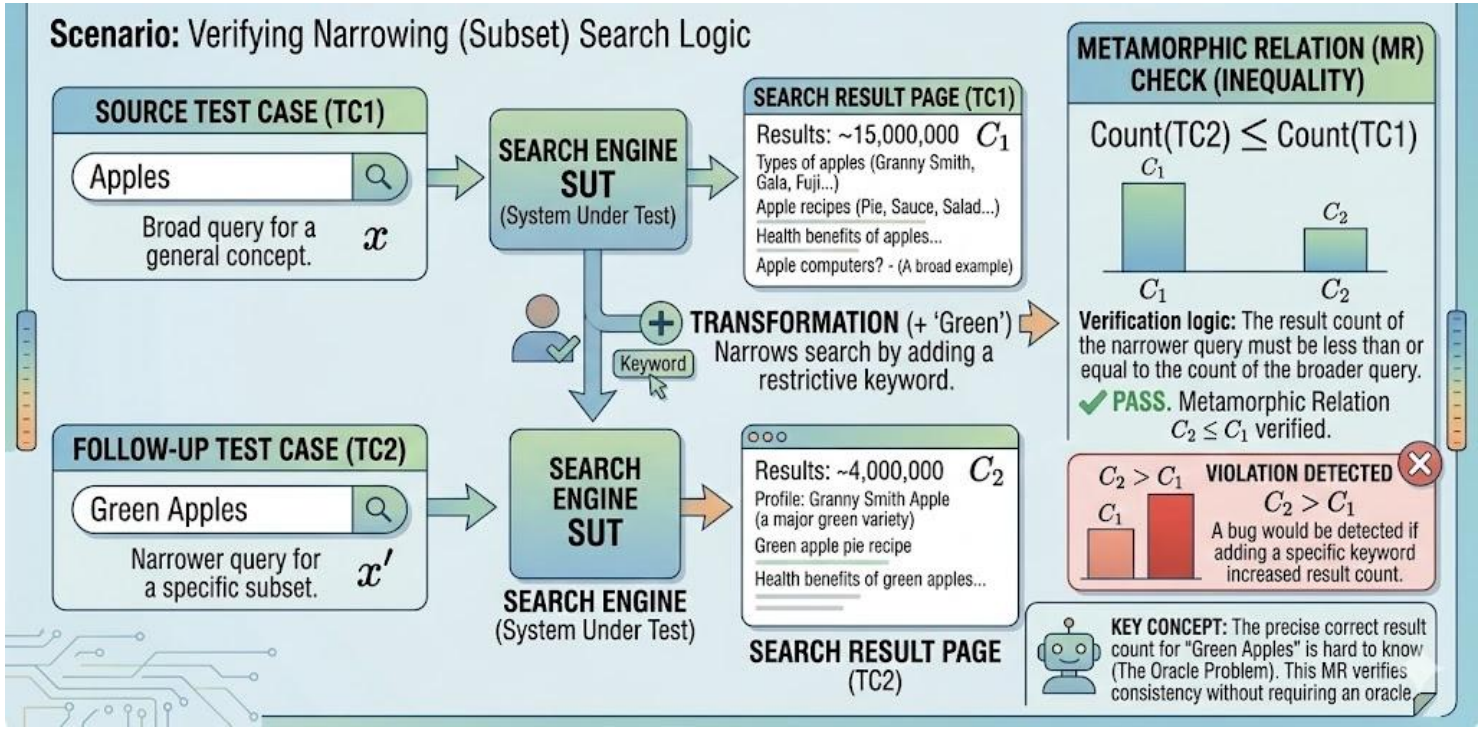


Example: Neural Network



(Image by Gemini)

Example: Search Engine



(Image by Gemini)

Test Oracle + Test Input

Follow-up test case that is inferred from a source test case that can come from various sources (e.g., randomly generated)

Metamorphic relation as a property to be verified

Test Oracle + Test Input

Metamorphic Testing at Cockroach Labs

"We employ a slew of testing methodologies and techniques. […] One such technique, known as ***metamorphic testing****, deserves more attention. We're* ***happy to vouch for it, given the number of obscure bugs it has helped us catch****, and also due to how exotic it sounds. […] We've caught more than our fair share of tasty fruit that has fallen from the metamorphic testing tree.* ***Many of the underlying bugs could never have been dreamt up during development****, even in the wildest dreams (or nightmares) of the developer. Many are very subtle edge cases, exercised with obscure database configurations. Many involve sequences of unanticipated operations.* ***Many have been severe, and would have resulted in corruption or data loss if they had made their way into a production binary****."*

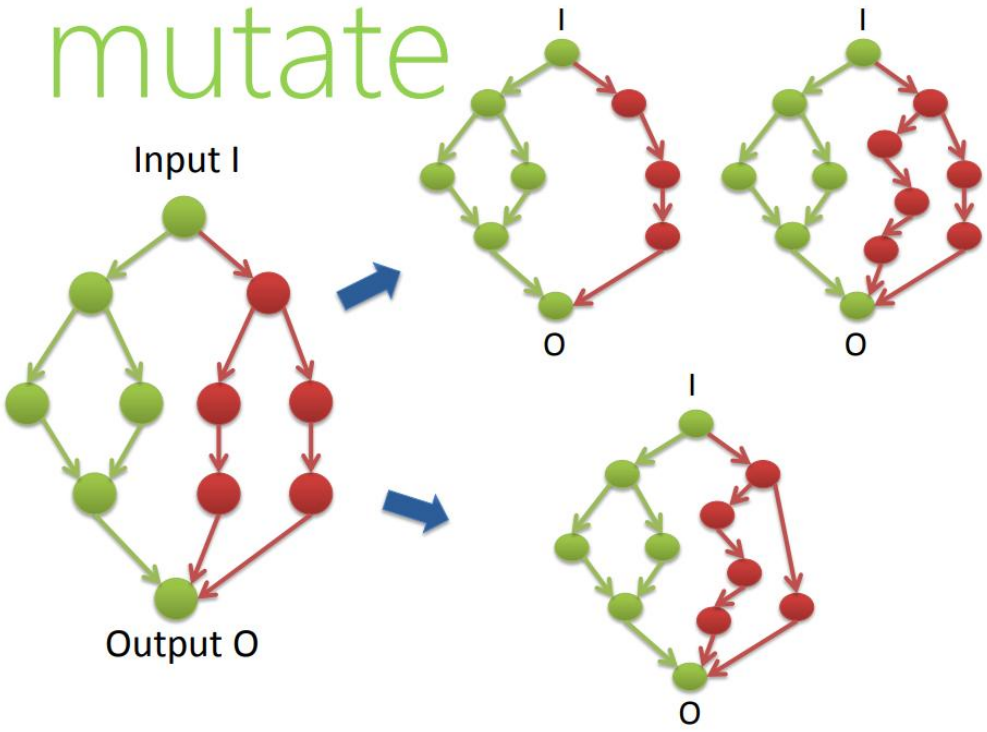
https://www.cockroachlabs.com/blog/metamorphic-testing-the-database/

EMI for Compiler Testing

- ***Equivalence Modulo Inputs (EMI)*** is one of the most successful metamorphic testing approaches and has found hundreds of bugs in GCC and LLVM
- Generating diverse **equivalent programs** for compiler testing is **challenging**
- Idea: rather than creating equivalent programs, create programs that are **equivalent only for a given input**, which is easier

https://people.inf.ethz.ch/suz/emi/index.html

EMI for Compiler Testing



https://people.inf.ethz.ch/suz/emi/index.html

EMI for Compiler Testing

- Approach
  - Compile and run a program P with an input I and records its output O=P(I)
  - **Mutate** the **unexecuted** part of P (e.g., by adding or deleting code), creating a program P’
    - P’ might have a different behavior as compared to P, but the same as compared to P(I)
  - Verify that P(I) = P’(I)

https://people.inf.ethz.ch/suz/emi/index.html

EMI for Compiler Testing

- Why can this technique find bugs?
  - The compiler needs to generate code **that produces the correct result** for ***any*** input, and optimizes both the executed and unexecuted parts
  - Already by only deleting code, this technique can find bugs in various compiler optimizations, assuming that the input source tests are diverse

EMI Illustration

```
int func(int a, int b, int c) {
    if (c) {
        return a + b + 5;
    } else {
        return a + b + 3;
    }
}
```

Compilers apply common subexpression elimination (CSE)

EMI Illustration

```
int func(int a, int b, int c) {
    int temp = a + b;
    if (c) {
        return temp + 5;
    } else {
        return temp + 3;
    }

}
```

Every optimization could possibly introduce bugs

EMI Illustration

```
int func(int a, int b, int c) {
    if (c) {
        return a + b + 5;
    } else {
        return a + b + 3;
    }
}
```

Assuming a = 1, b = 2, c = 3, we know that the else branch is not executed

EMI Illustration

```
int func(int a, int b, int c) {
    if (c) {
        return a + b + 5;
    } else {
        return a + c + 3;
    }
}
```

If we mutate it, CSE might no longer be applicable

Metamorphic Testing Characteristics

- Strengths
  - Avoids some of the weaknesses in differential testing, such as requiring multiple systems that produce the same result for a given input: metamorphic testing requires only a single system
  - Simple and effective technique (assuming that the metamorphic relation is simple)
- Weaknesses
  - Finding a bug-revealing metamorphic relation can be difficult
  - Overlooks bugs where the system acts consistently incorrect

Summary and Takeaways

- Metamorphic testing
- Metamorphic relation
- Equivalence Modulo Inputs (EMI)